Kako računala igraju šah?

Matija Njegač

PMF — MO

5. studenoga 2026.

Antagonističke igre

- Antagonističke igre su one koje se sastoje od dva ili više igrača koji imaju suprotne ciljeve. Pobjeda jednog igrača znači identičan i absolutni gubitak za ostale igrače.
- Primjeri: šah, križić kružić, go, shogi

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

- **Korijen** $v_0 \in V$**:** Početno stanje igre s_0.

[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

- **Korijen** $v_0 \in V$**:** Početno stanje igre s_0.
- **Čvorovi** V**:** Pozicije. Svakom neterminalnom čvoru pridružen je igrač na potezu $P(v) \in \{\text{MAX}, \text{MIN}\}$.

[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

- **Korijen** $v_0 \in V$**:** Početno stanje igre s_0.
- **Čvorovi** V**:** Pozicije. Svakom neterminalnom čvoru pridružen je igrač na potezu $P(v) \in \{\text{MAX}, \text{MIN}\}$.
- **Usmjereni bridovi** E**:** Legalni potezi

[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

- **Korijen** $v_0 \in V$**:** Početno stanje igre s_0.
- **Čvorovi** V**:** Pozicije. Svakom neterminalnom čvoru pridružen je igrač na potezu $P(v) \in \{\text{MAX}, \text{MIN}\}$.
- **Usmjereni bridovi** E**:** Legalni potezi
- **Listovi (terminalni čvorovi)** $V_T \subset V$**:** Završne pozicije (mat, pat, remi) gdje je definirana funkcija isplate:

$$f : V_T \to \mathbb{R} \quad \text{(npr. Shannon ju definira kao:} \{1, 0, -1\})^1$$

[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Stablo igre

Deterministička igru dvaju igrača s potpunom informacijom prikazujemo usmjerenim stablom $T = (V, E)$:

- **Korijen** $v_0 \in V$**:** Početno stanje igre s_0.
- **Čvorovi** V**:** Pozicije. Svakom neterminalnom čvoru pridružen je igrač na potezu $P(v) \in \{\text{MAX}, \text{MIN}\}$.
- **Usmjereni bridovi** E**:** Legalni potezi
- **Listovi (terminalni čvorovi)** $V_T \subset V$**:** Završne pozicije (mat, pat, remi) gdje je definirana funkcija isplate:

$$f : V_T \to \mathbb{R} \quad \text{(npr. Shannon ju definira kao:} \{1, 0, -1\})^1$$

Svaki put od korijena do lista predstavlja jednu cjelovitu partiju.

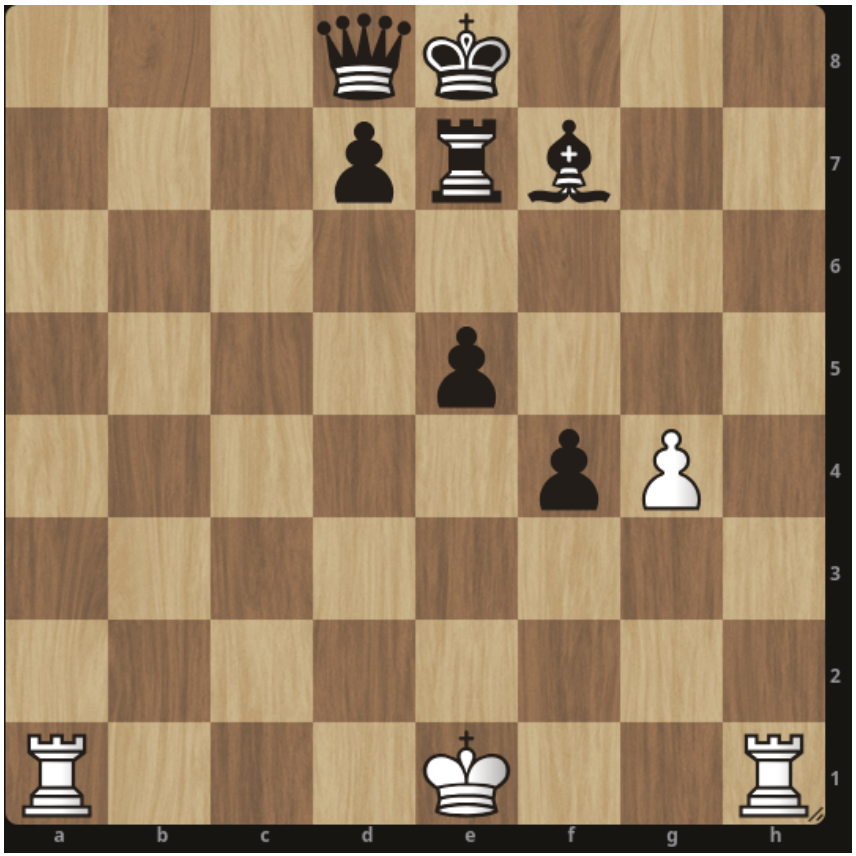
[1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, 1950.

Što je šahovska pozicija?

Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?

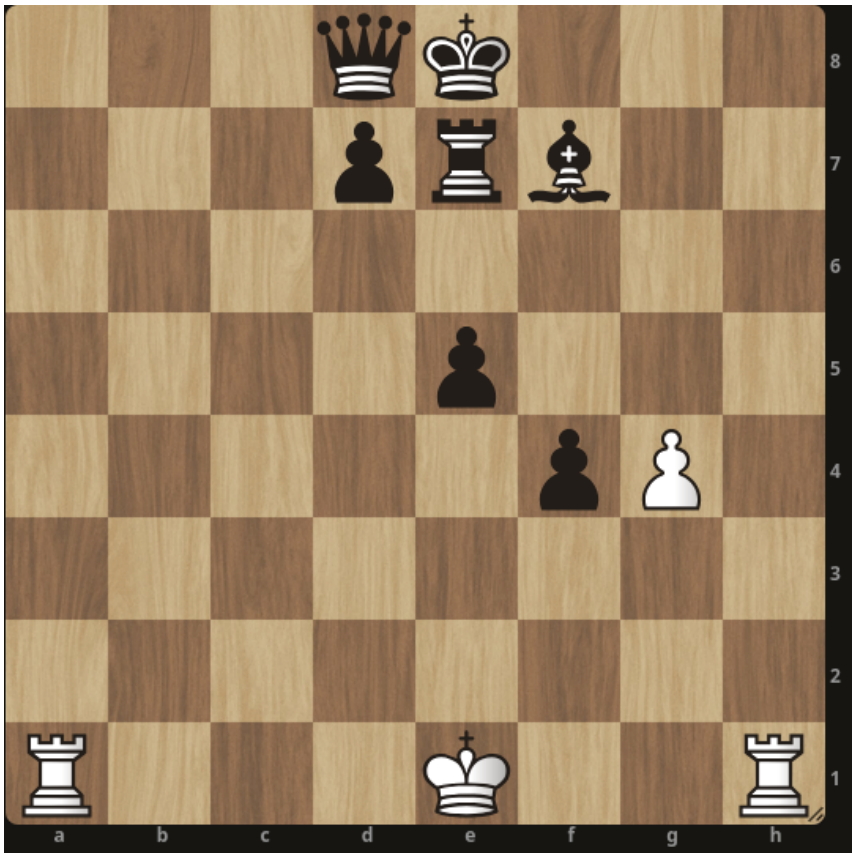
Što je šahovska pozicija?

Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



Što je šahovska pozicija?

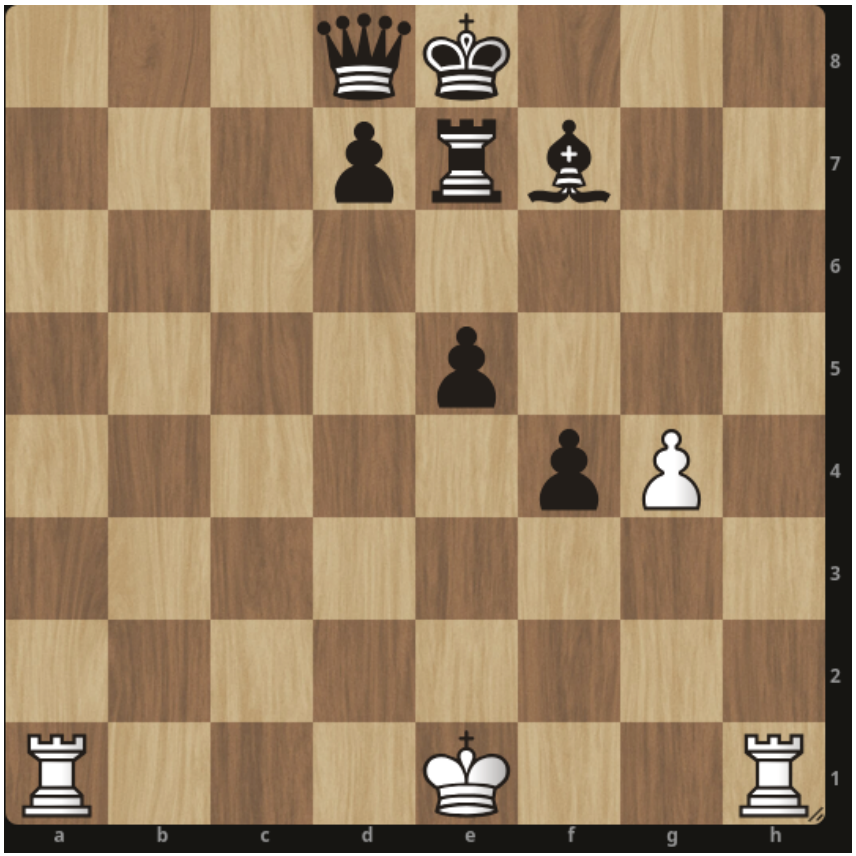
Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



- Položaj figura

Što je šahovska pozicija?

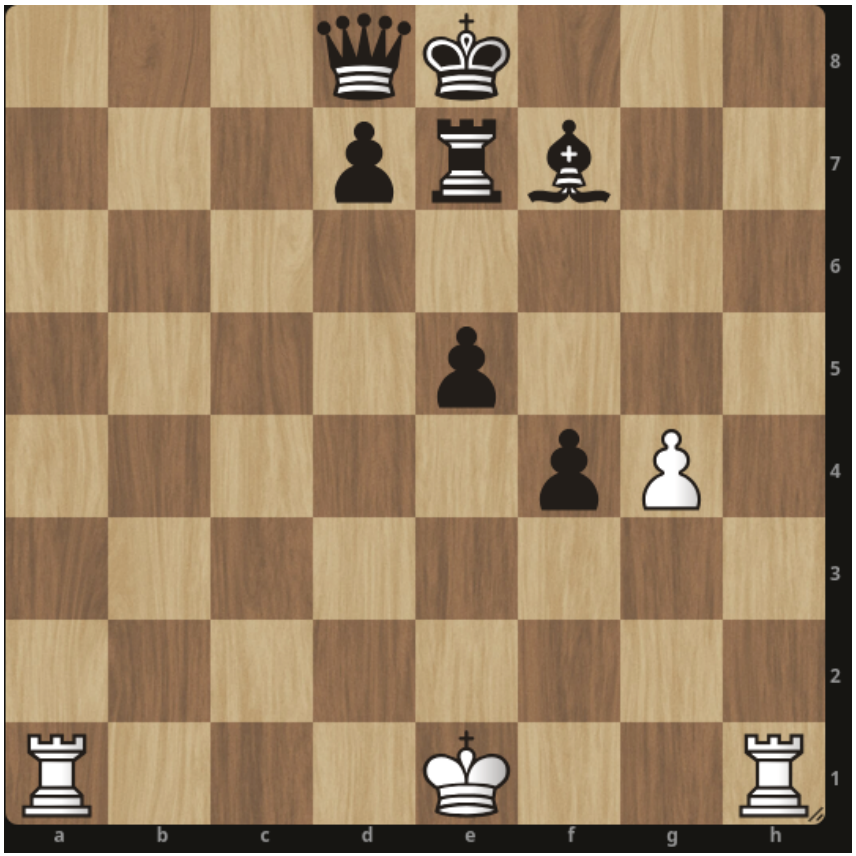
Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



- Položaj figura
- Tko je na potezu

Što je šahovska pozicija?

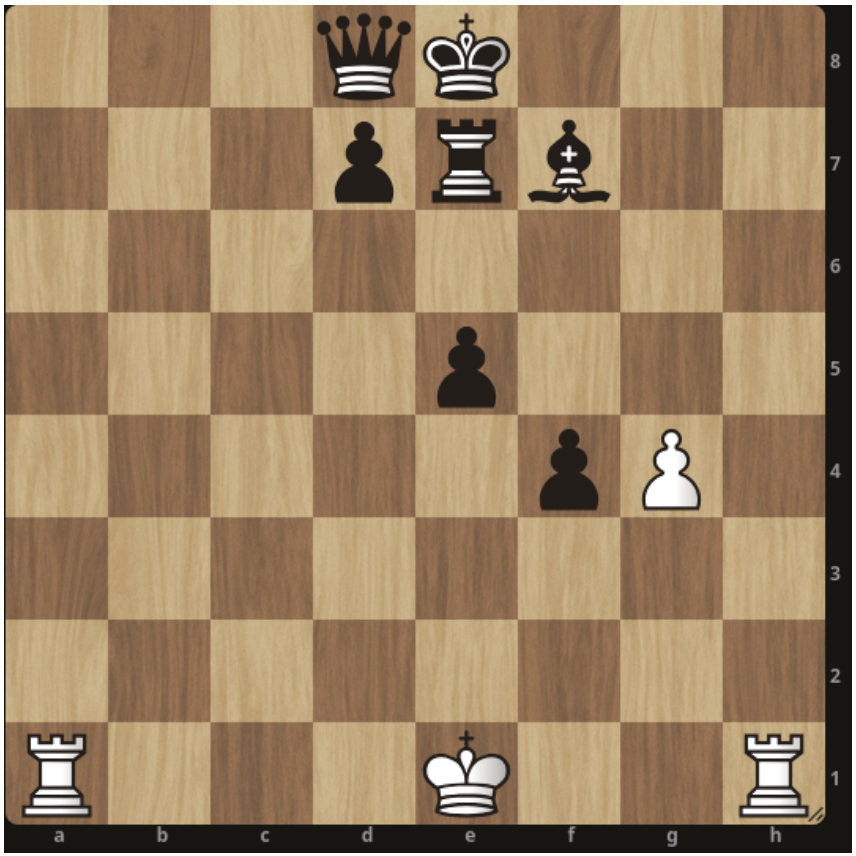
Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



- Položaj figura
- Tko je na potezu
- Jesu li se micali kralj ili topovi

Što je šahovska pozicija?

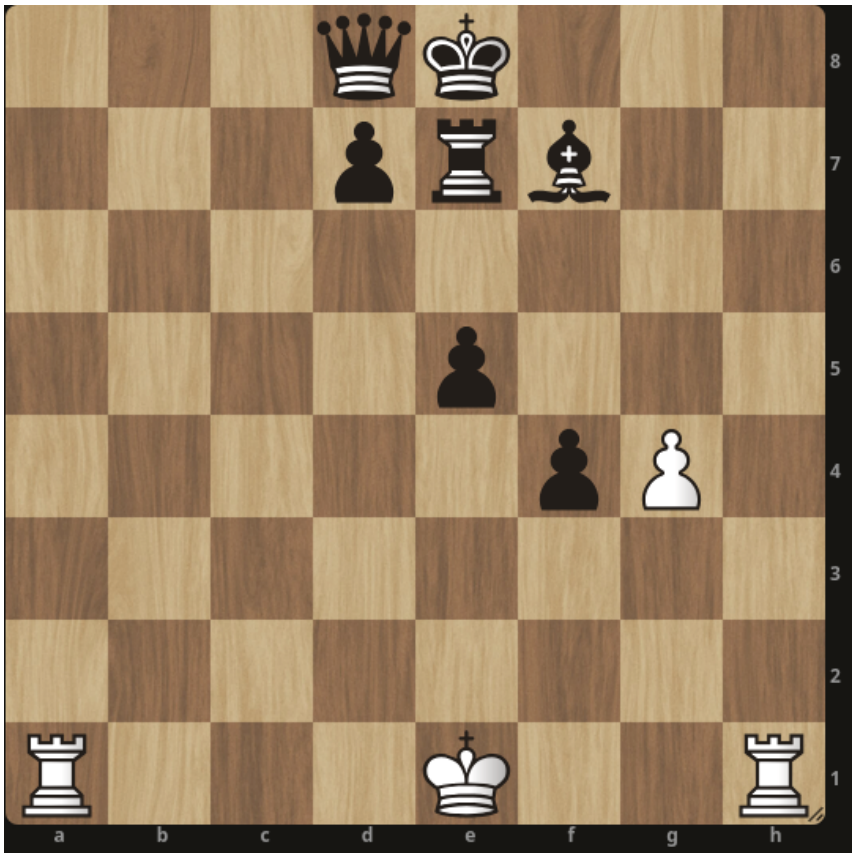
Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



- Položaj figura
- Tko je na potezu
- Jesu li se micali kralj ili topovi
- Je li moguć en passant

Što je šahovska pozicija?

Koje uvjete trebamo zadovoljiti da jednoznačno definiramo šahovsku poziciju?



- Položaj figura
- Tko je na potezu
- Jesu li se micali kralj ili topovi
- Je li moguć en passant
- Koliko poteza je prošlo od zadnjeg uzimanja figure ili pomicanja pješaka

Najčešći prikaz za zapis pozicije koji se koristi je Forsyth-Edwards Notacija (FEN) zapis

Primjer FEN zapisa

Primjer FEN zapisa:

`2r1k2r/1pQ1nppp/p1b1p1q1/8/4P3/2N1B1P1/PPPR1P1P/2KR4 b k - 0 15`



Primjer FEN zapisa

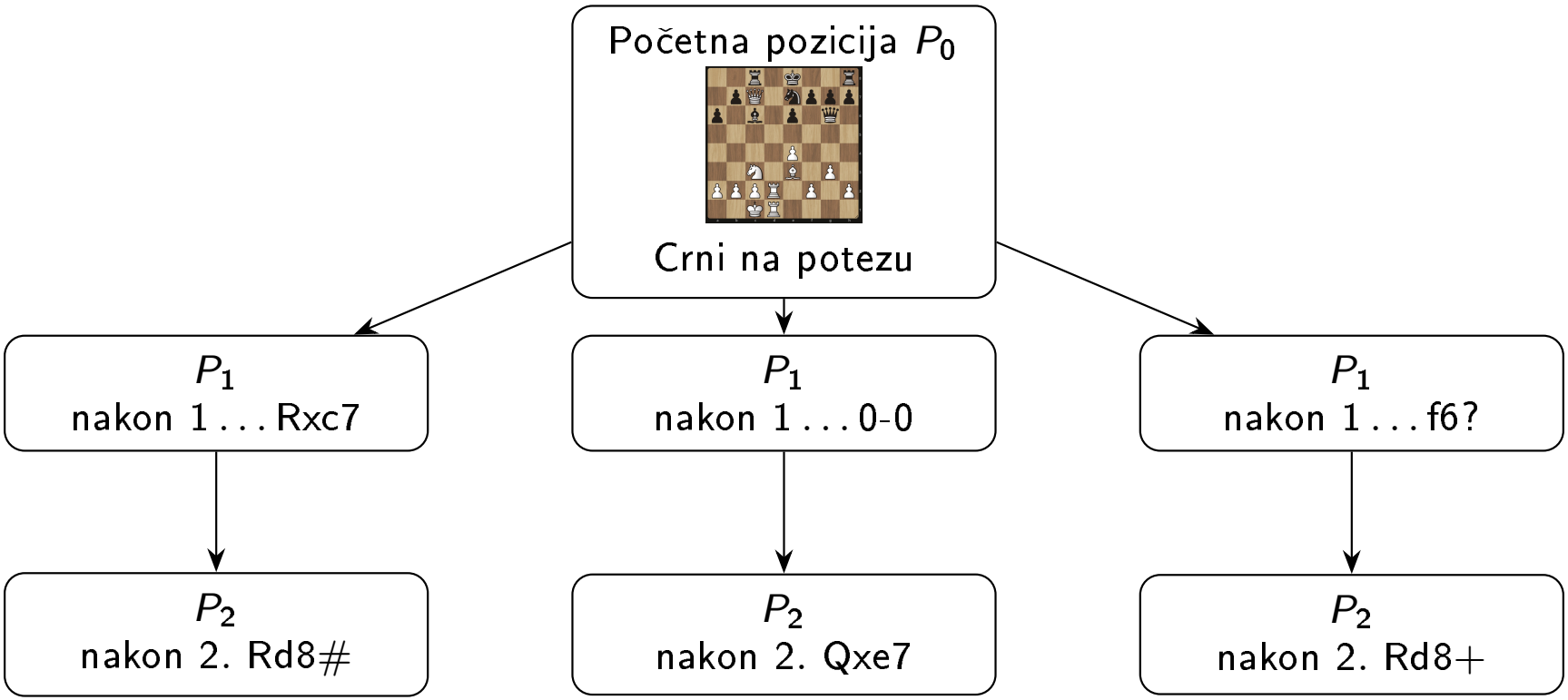
Primjer FEN zapisa:

`2r1k2r/1pQ1nppp/p1b1p1q1/8/4P3/2N1B1P1/PPPR1P1P/2KR4 b k - 0 15`



Njegač – Tuković, Split 2021.

Pojednostavljeno stablo igre



Nisu prikazani svi legalni potezi.

Problem?

- **Koliko postoji pozicija?**

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*
- **Koliko čvorova u stablu?**

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*
- **Koliko čvorova u stablu?**
  - Prosječni broj legalnih poteza: $b \approx 30$

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*
- **Koliko čvorova u stablu?**
  - Prosječni broj legalnih poteza: $b \approx 30$
  - Prosječni broj (polu)poteza u jednoj partiji: $d \approx 80$

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*
- **Koliko čvorova u stablu?**
  - Prosječni broj legalnih poteza: $b \approx 30$
  - Prosječni broj (polu)poteza u jednoj partiji: $d \approx 80$
  - $b^d = 30^{80} \approx 10^{120}$ (Shannonov broj)

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Problem?

- **Koliko postoji pozicija?**
  - 4.822×10^{44}*
- **Koliko čvorova u stablu?**
  - Prosječni broj legalnih poteza: $b \approx 30$
  - Prosječni broj (polu)poteza u jednoj partiji: $d \approx 80$
  - $b^d = 30^{80} \approx 10^{120}$ (Shannonov broj)

PREVIŠE!

*J. Tromp, *Counting Chess Positions*, ICGA Journal 44(2), 2022.

Rješenje?

Rješenje?

Funkcija statičke evaluacije:

$$Eval : V \to \mathbb{R}$$

Svakom čvoru dodjeljujemo realni broj gdje pozitivan broj označava prednost bijelog, a negativan prednost crnog.

Rješenje?

Funkcija statičke evaluacije:

$$Eval\colon V \to \mathbb{R}$$

Svakom čvoru dodjeljujemo realni broj gdje pozitivan broj označava prednost bijelog, a negativan prednost crnog.

Definiranje takve funkcije je umjetnost:

Shannon (1950)

$$\begin{aligned} Eval(P) = 200(K-K') + 9(Q-Q') + 5(R-R') + 3(B-B'+N-N') + (P-P') \\ - 0.5(D-D'+S-S'+I-I') + 0.1(M-M') \end{aligned}$$

Rješenje?

Funkcija statičke evaluacije:

$$Eval\colon V \to \mathbb{R}$$

Svakom čvoru dodjeljujemo realni broj gdje pozitivan broj označava prednost bijelog, a negativan prednost crnog.

Definiranje takve funkcije je umjetnost:

Shannon (1950)

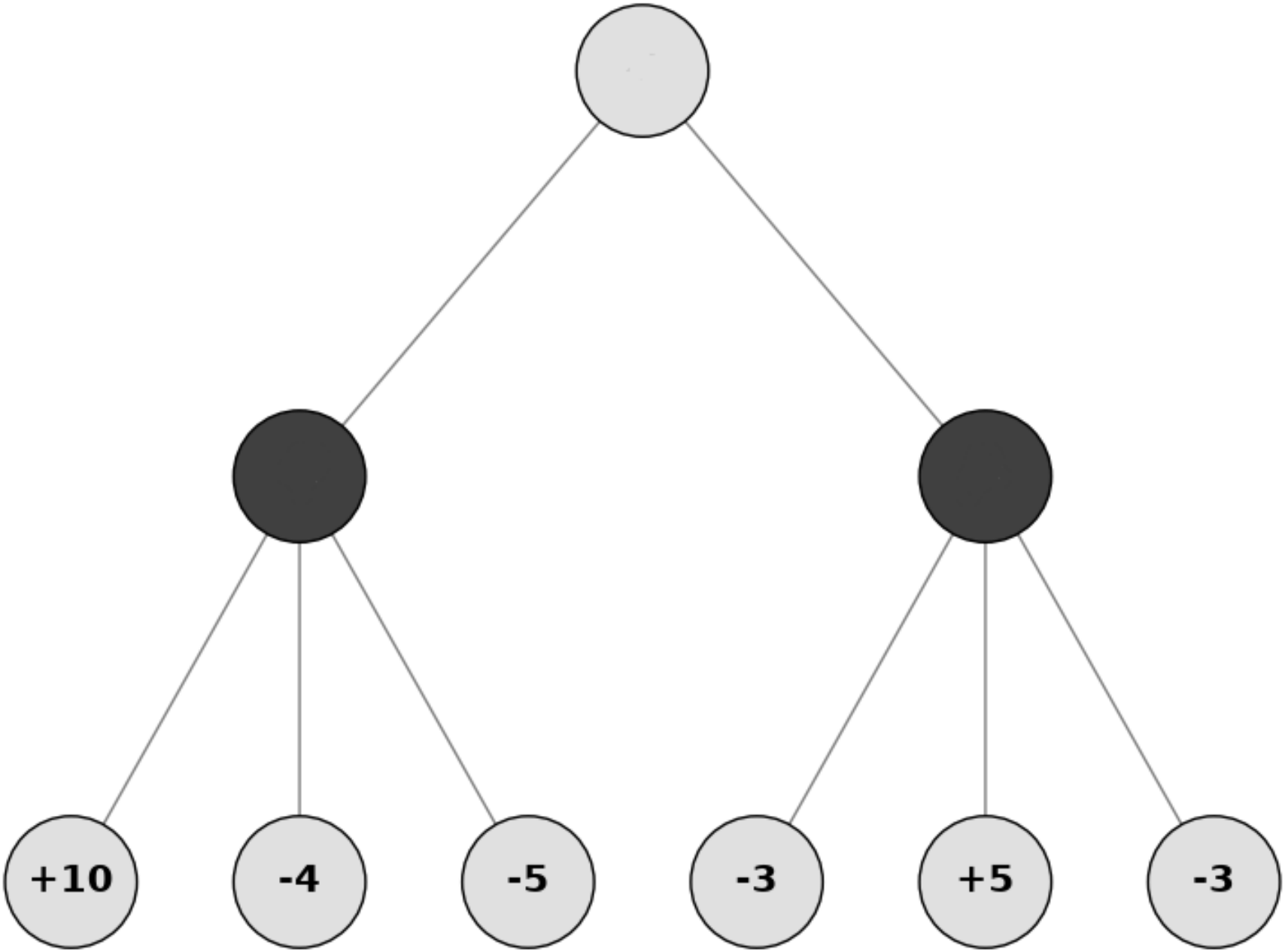
$$\begin{aligned} Eval(P) = 200(K-K') + 9(Q-Q') + 5(R-R') + 3(B-B'+N-N') + (P-P') \\ - 0.5(D-D'+S-S'+I-I') + 0.1(M-M') \end{aligned}$$

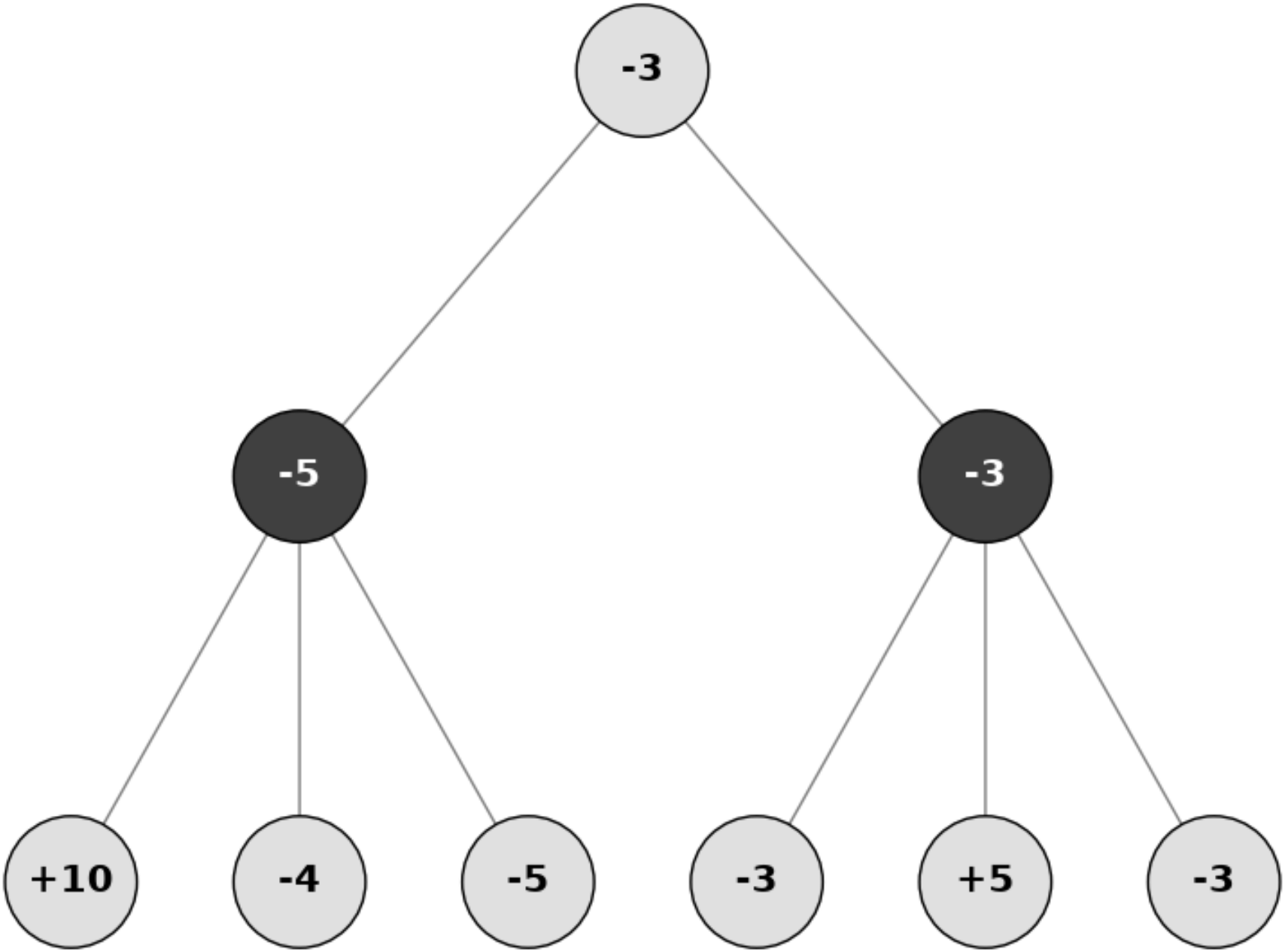
Desetljećima su inženjeri ručno definirali evaluacijske funkcije:

- **Deep Blue (1997):** 8000 ručno kodiranih parametara

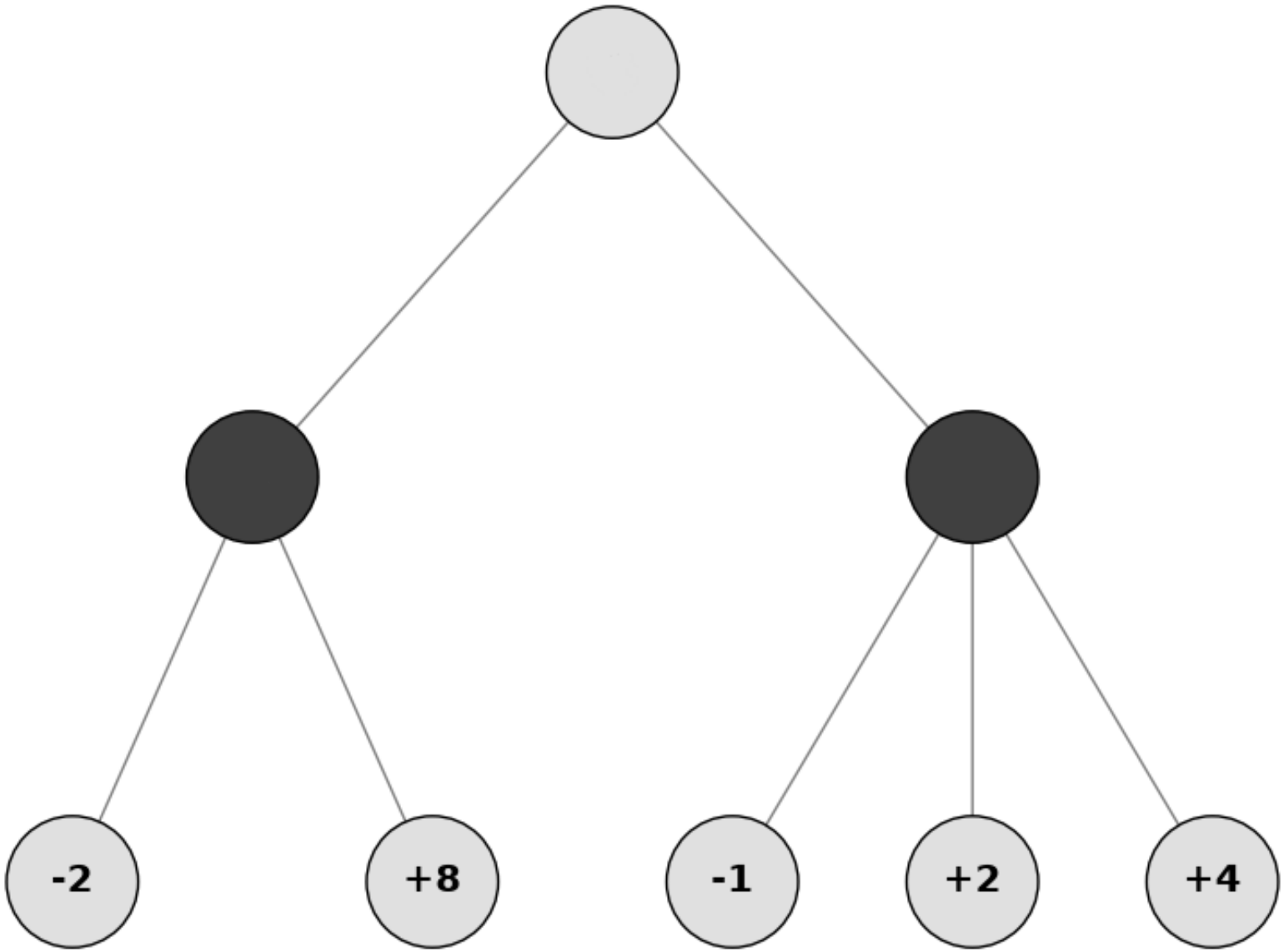
Minimax algoritam

- Za minimax radi na način da uzmemo dubinu d traženja.
- sve čvorove na dubini d evaluiramo funkcijom statičke evaluacije
- ovisno o igraču pronađemo minimum ili maximum sve djece čvora i njemu pridružujemo tu vrijednost


+10
-4
-5
-3
+5
-3


-3
-5
-3
+10
-4
-5
-3
+5
-3

Zadatak



Minimax algoritam

```
1  BIJELA = 1
2  CRNA = 0
3
4  class Stablo:
5      def __init__(self, boja: bool,
6                   evaluacija: Optional[float] = None):
7          self.boja = boja
8          self.evaluacija = evaluacija
9          self.djeca: list['Stablo'] = []
10
11     def dodaj_dijete(self, dijete: 'Stablo'):
12         self.djeca.append(dijete)
13
14     def daj_boju(self):
15         return self.boja
16
17     def daj_evaluaciju(self):
18         return self.evaluacija
19
20     def postavi_evaluaciju(self, eval):
21         self.evaluacija = eval
22     def je_list(self):
23         return len(self.djeca) == 0
24
25 def staticka_evaluacija(pozicija: Stablo):
26     return random.uniform(-10, 10)
```

Minimax algoritam

```
1  def minimax(pozicija: Stablo, dubina: int, bojaIgraca: bool):
2      najbolje_dijete = None
3      if dubina == 0 or pozicija.je_list():
4          if pozicija.evaluacija is None:
5              pozicija.evaluacija = staticka_evaluacija(pozicija)
6          return pozicija.evaluacija, None
7
8      if bojaIgraca == BIJELA:
9          najbolja_evaluacija = float('-inf')
10         for dijete in pozicija.djeca:
11             vrijednost, _ = minimax(dijete, dubina - 1, CRNA)
12             if vrijednost > najbolja_evaluacija:
13                 najbolja_evaluacija = vrijednost
14                 najbolje_dijete = dijete
15     else:
16         najbolja_evaluacija = float('inf')
17         for dijete in pozicija.djeca:
18             vrijednost, _ = minimax(dijete, dubina - 1, BIJELA)
19             if vrijednost < najbolja_evaluacija:
20                 najbolja_evaluacija = vrijednost
21                 najbolje_dijete = dijete
22
23     pozicija.evaluacija = najbolja_evaluacija
24     return najbolja_evaluacija, najbolje_dijete
```

Može li bolje?

Minimax izračunava sve čvorove. Mora li ih sve?

α-β **podrezivanje**:

- α: Najveći ishod koji igrač MAX (bijeli) ima osiguran. (init $= -\infty$)
- β: Najmanji ishod koji igrač MIN (crni) ima osiguran. (init $= inf$)

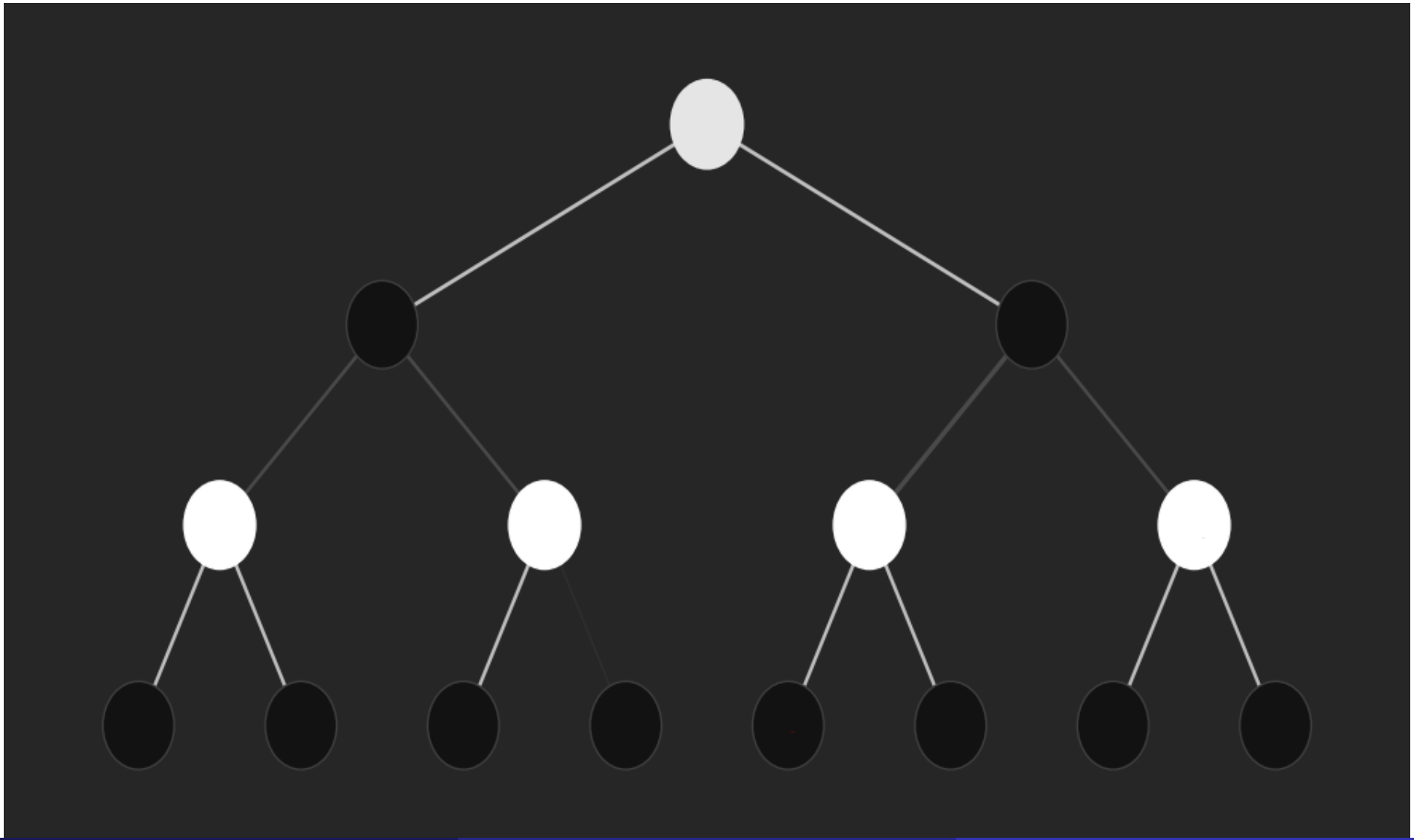
Čim u nekom čvoru vrijedi:

$$\beta \leq \alpha$$

prekidamo pretragu grane. Ta grana nikad neće biti bolja opcija!

Primjer

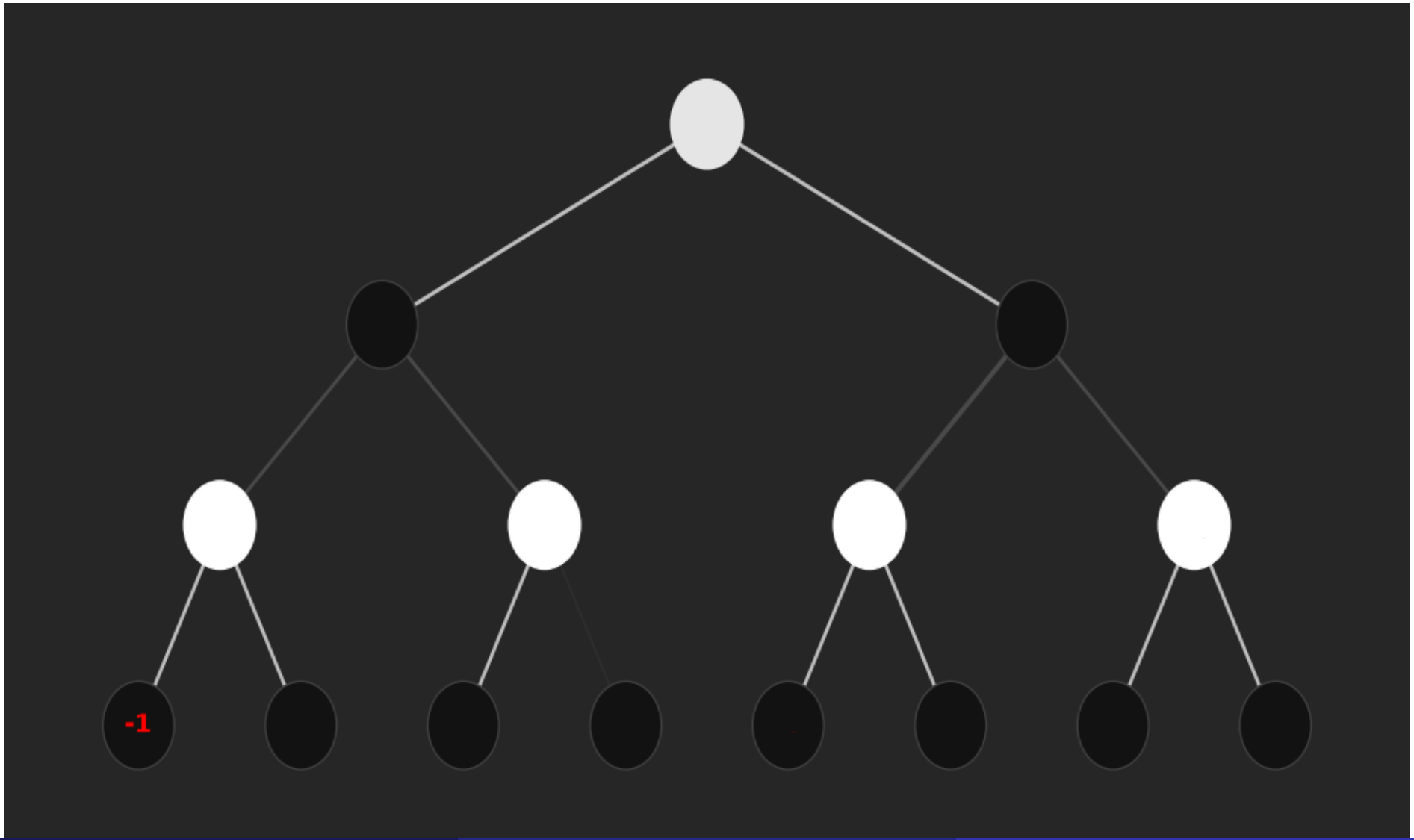
$$\alpha = -\infty \; \beta = \infty$$
$$\beta > \alpha$$



Primjer

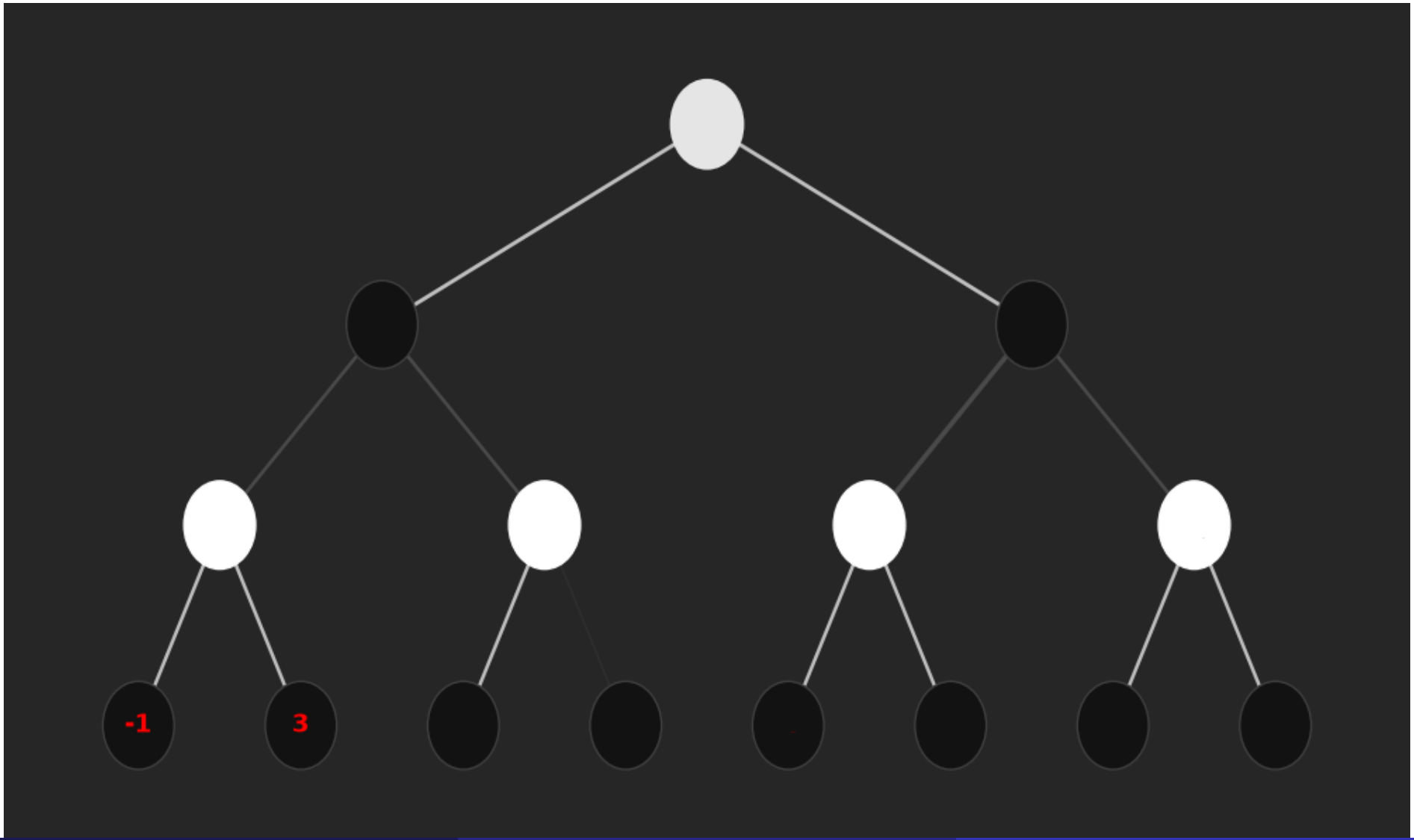
$$\alpha = -1 \; \beta = \infty$$

$$\beta > \alpha$$



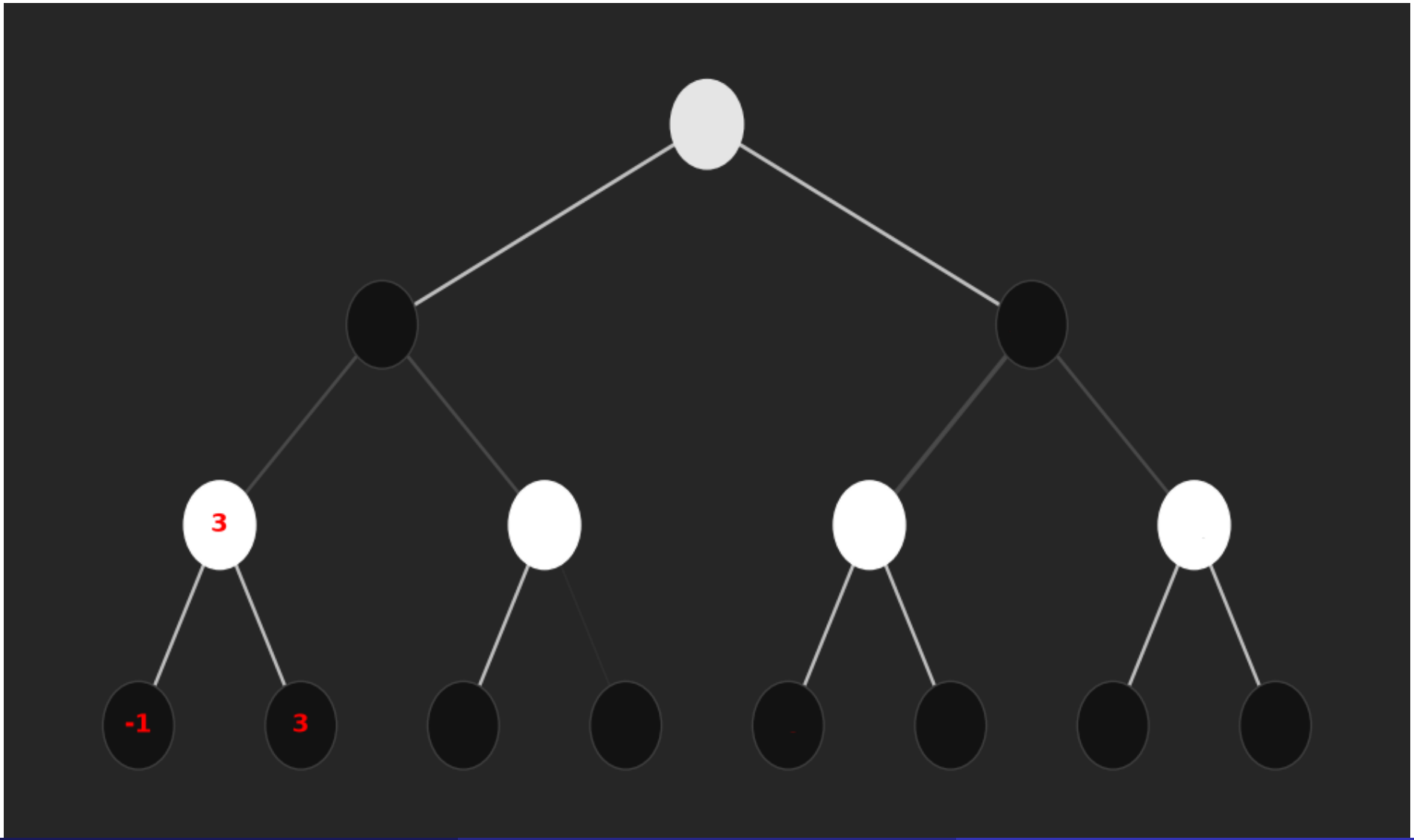
Primjer

$$\alpha = 3 \; \beta = \infty$$
$$\beta > \alpha$$



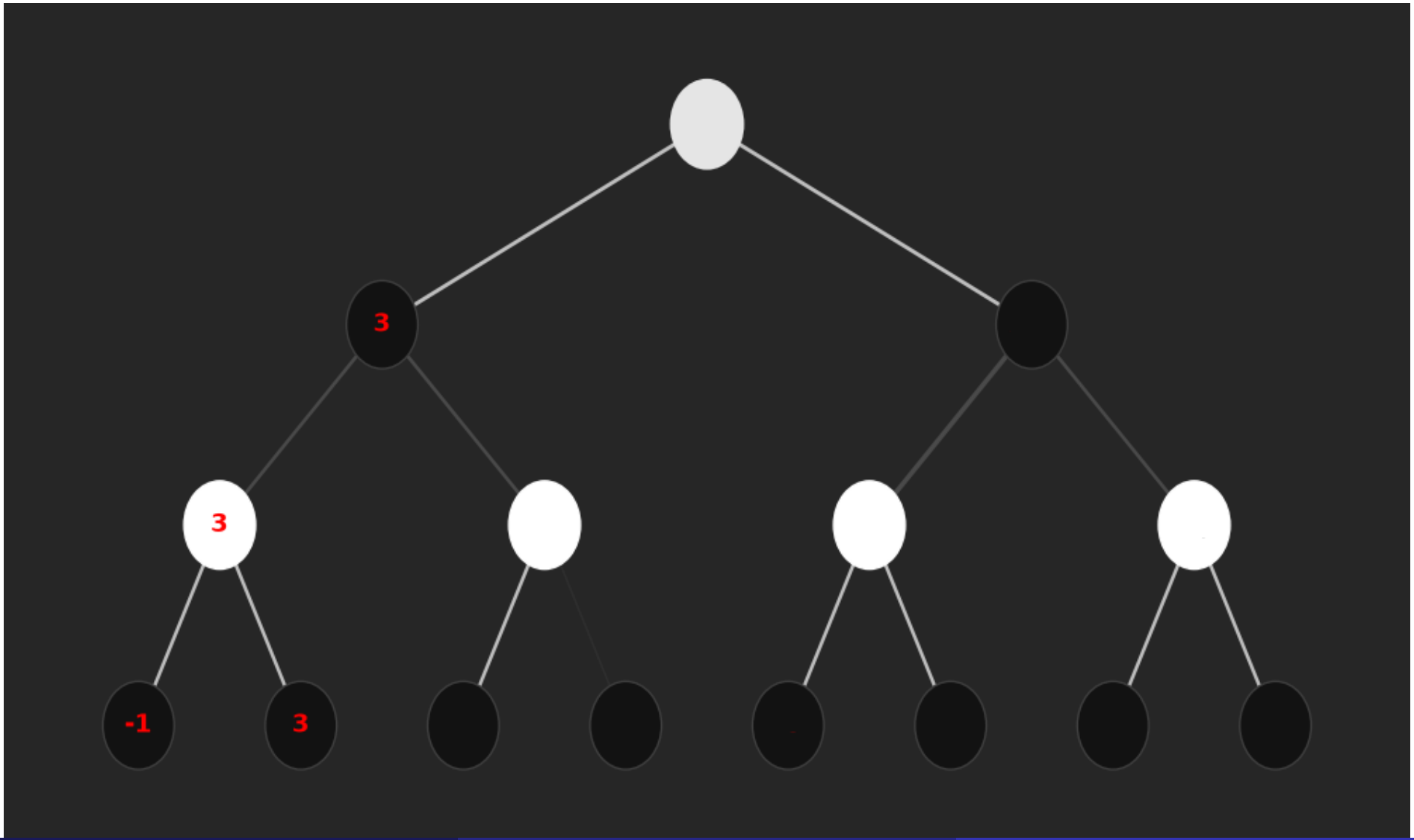
Primjer

$$\alpha = 3 \; \beta = \infty$$
$$\beta > \alpha$$



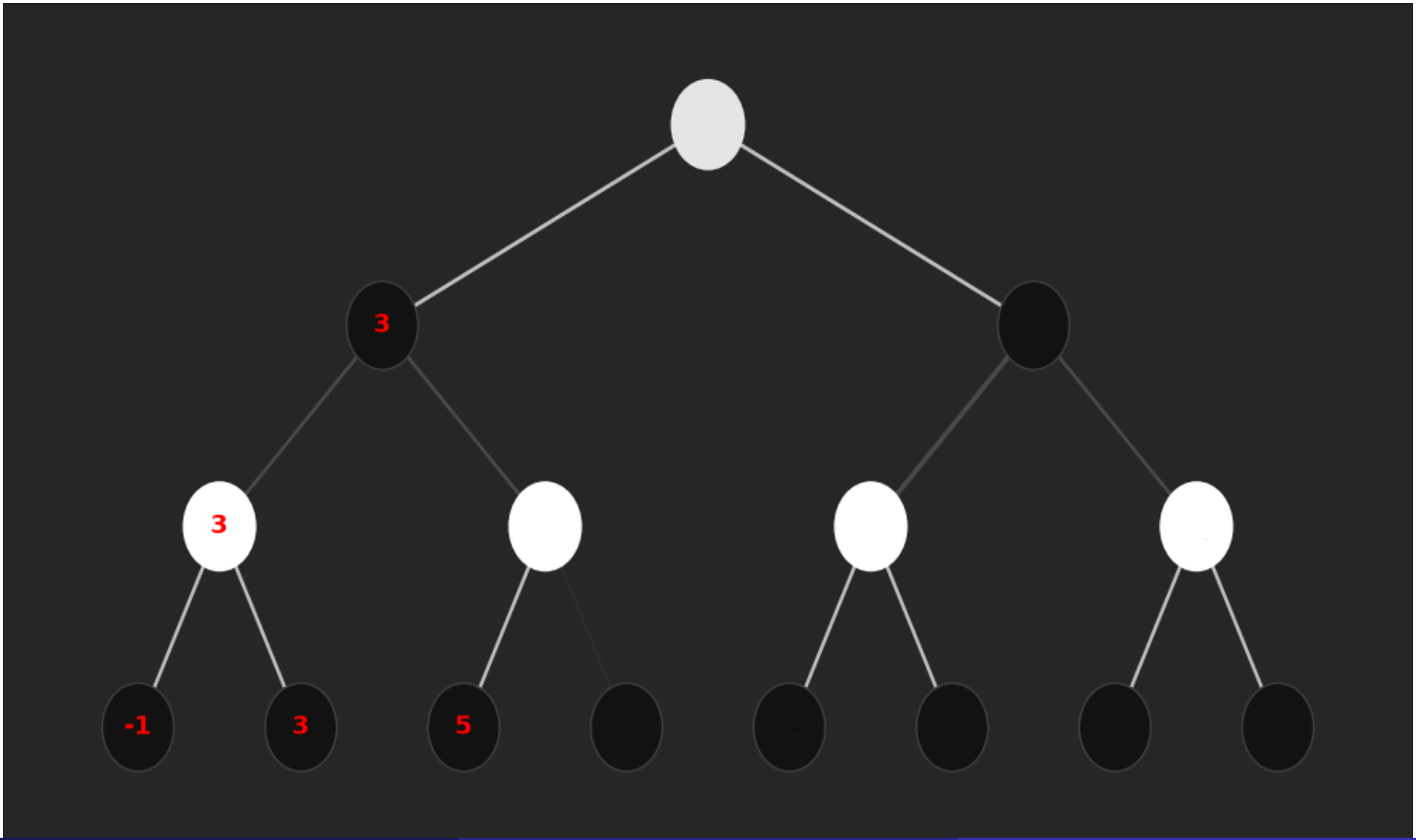
Primjer

$$\alpha = -\infty \ \beta = 3$$
$$\beta > \alpha$$



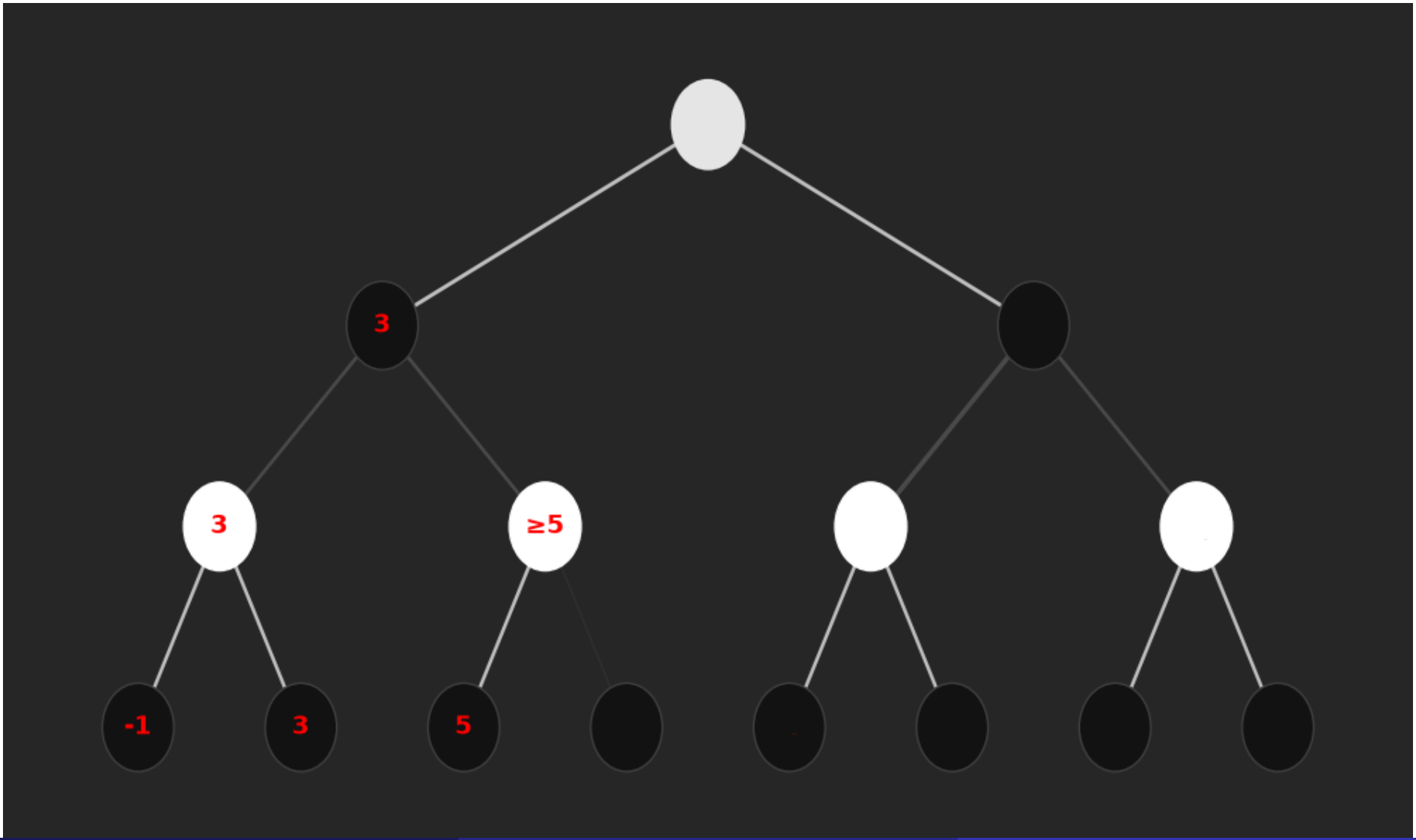
Primjer

$$\alpha = 5 \ \beta = 3$$
$$\beta \leq \alpha \text{ (rez!)}$$



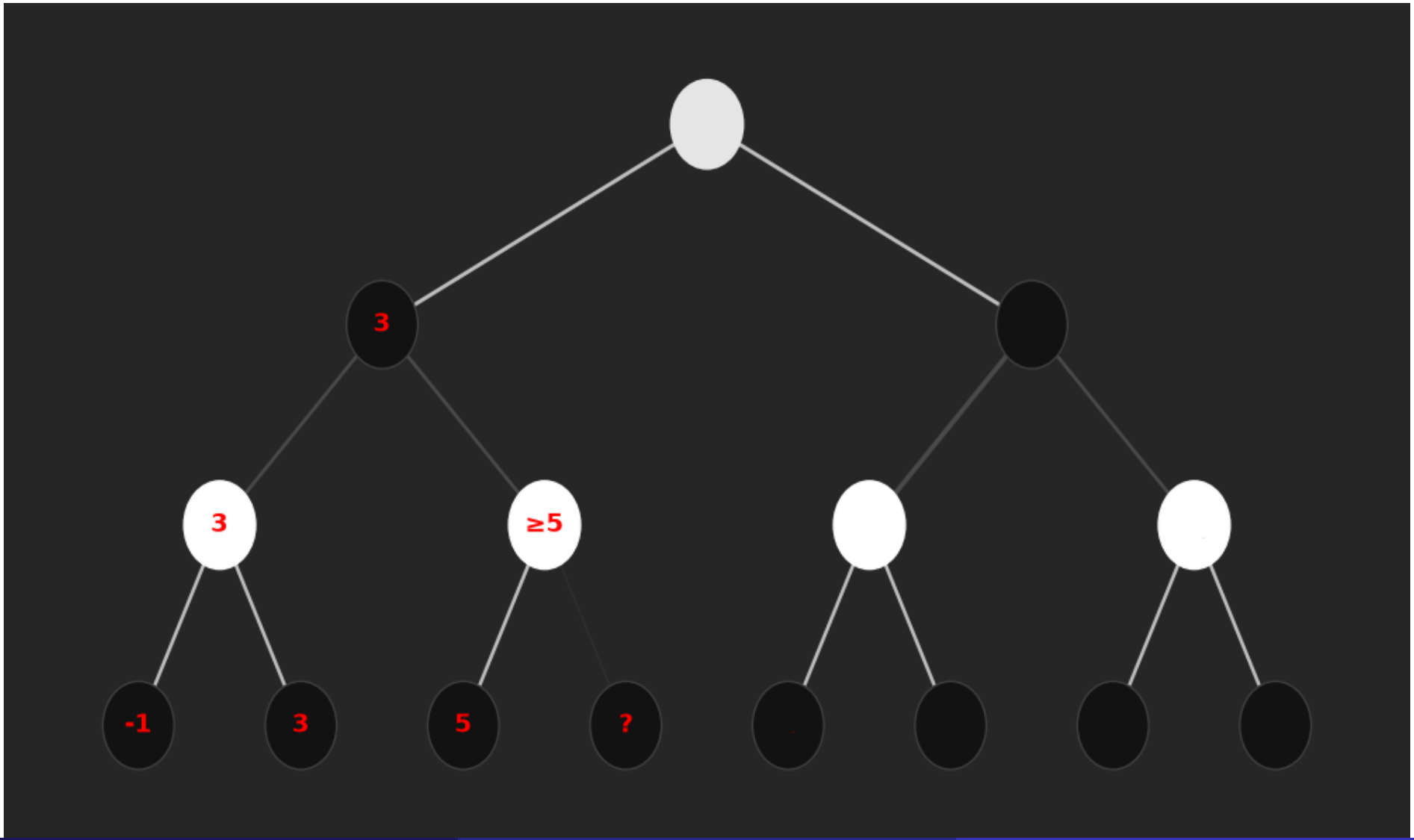
Primjer

$$\alpha = 5 \ \beta = 3$$
$$\beta \leq \alpha \text{ (rez!)}$$



Primjer

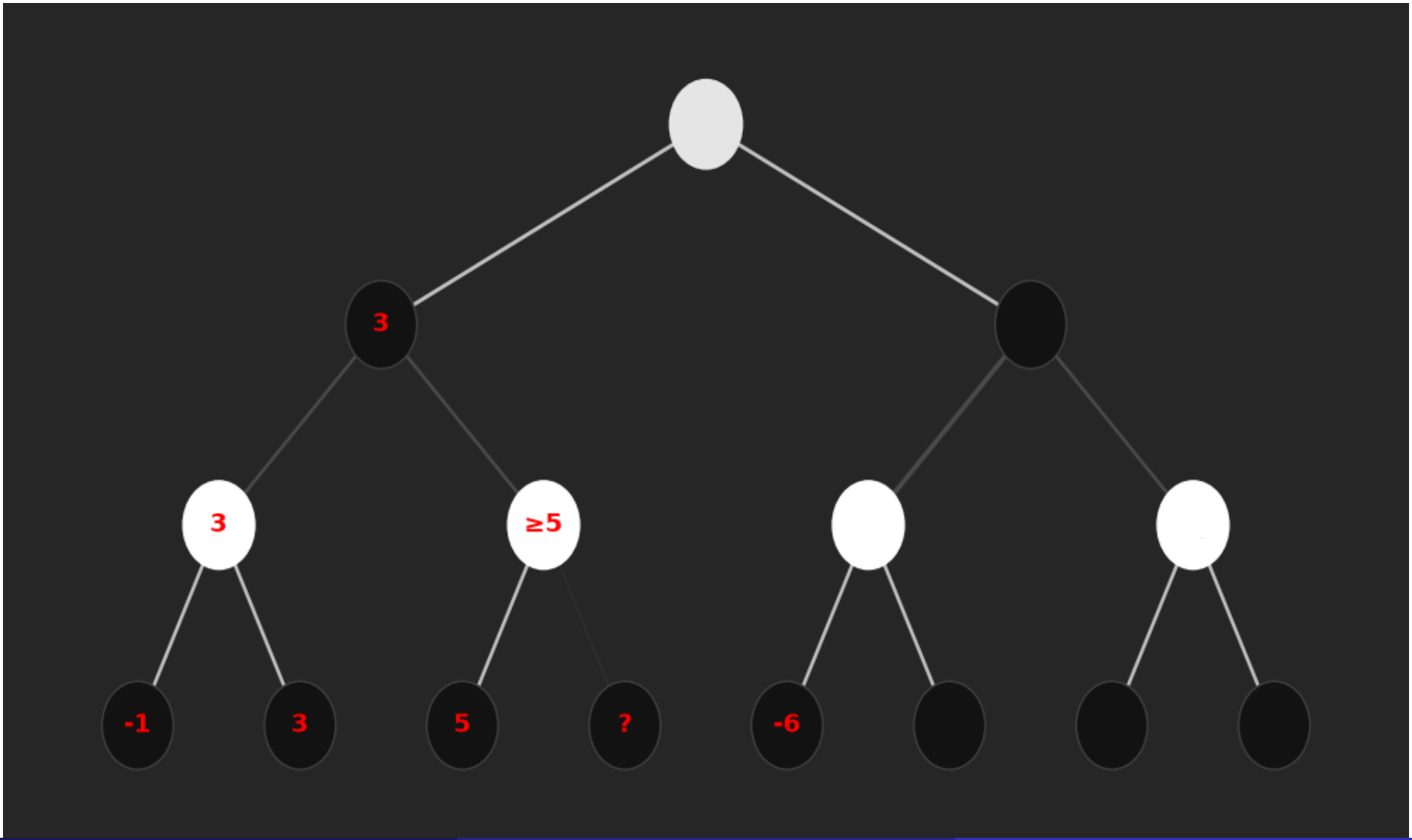
$$\alpha = 3 \; \beta = \infty$$
$$\beta > \alpha$$



Primjer

$$\alpha = 3 \; \beta = \infty$$

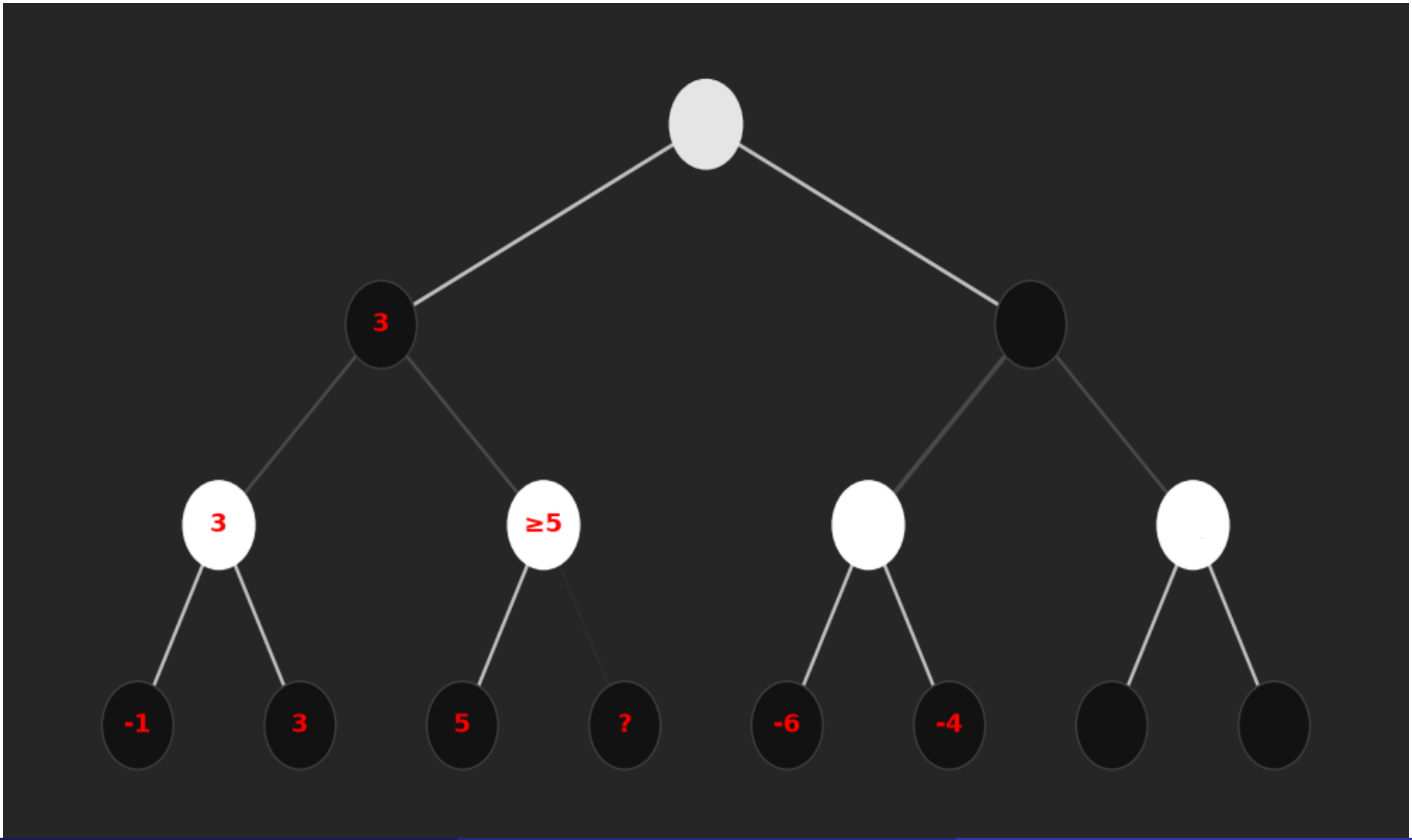
$$\beta > \alpha$$



Primjer

$$\alpha = 3 \; \beta = \infty$$

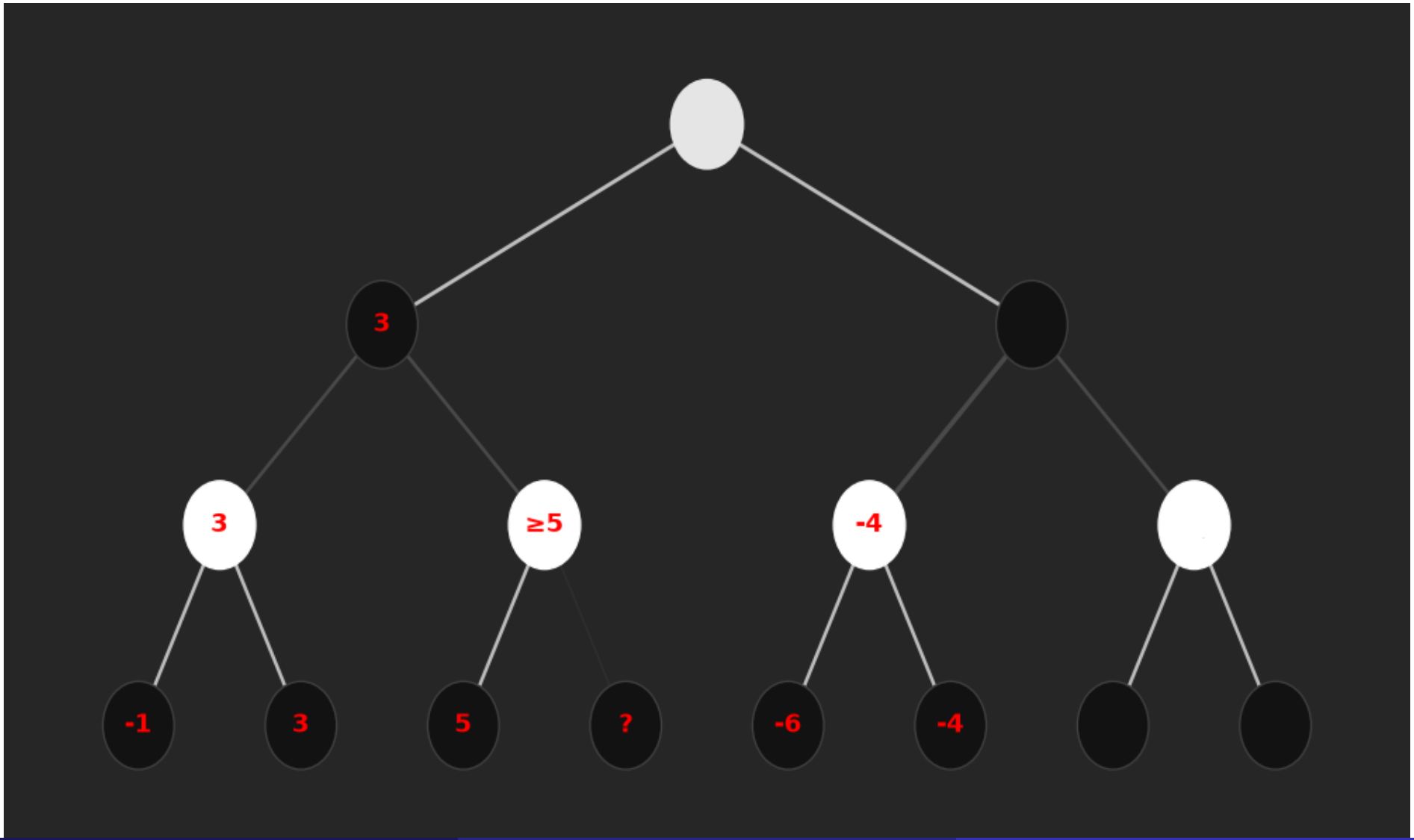
$$\beta > \alpha$$



Primjer

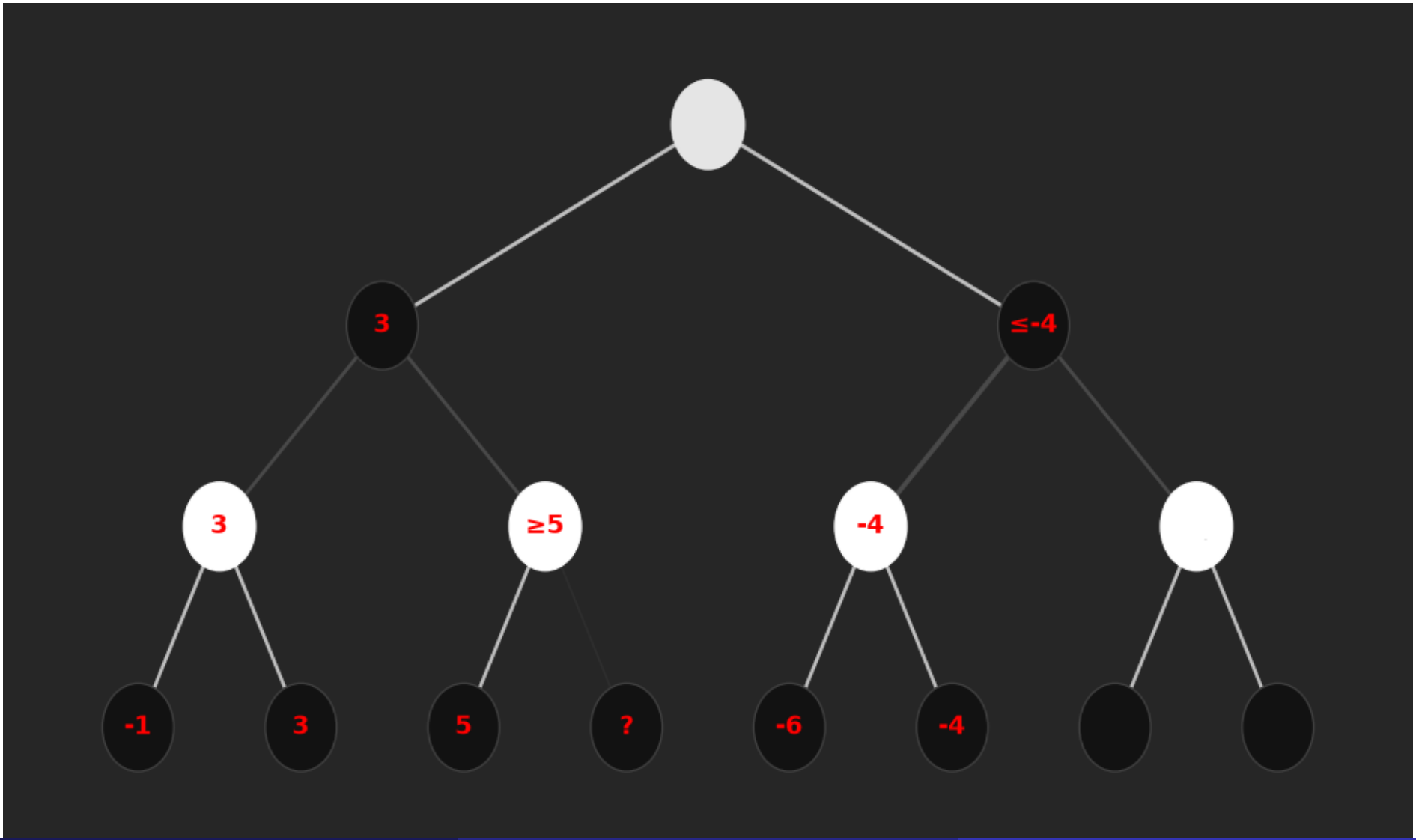
$$\alpha = 3 \; \beta = \infty$$

$$\beta > \alpha$$



Primjer

$$\alpha = 3 \; \beta = -4$$
$$\beta \leq \alpha \text{ (rez!)}$$



Implementacija: α-β podrezivanje

```
1   def alfa_beta_podrezivanje(pozicija: Stablo, dubina: int, bojaIgraca: bool,
2                              alpha: float = float('-inf'),
3                              beta: float = float('inf')):
4       najbolje_dijete = None
5       if dubina == 0 or pozicija.je_list():
6           if pozicija.evaluacija is None:
7               pozicija.evaluacija = staticka_evaluacija(pozicija)
8           return pozicija.evaluacija, None
9
10      if bojaIgraca == BIJELA:
11          najbolja_evaluacija = float('-inf')
12          for dijete in pozicija.djeca:
13              vrijednost, _ = alfa_beta_podrezivanje(dijete, dubina-1, CRNA, alpha, beta)
14              if vrijednost > najbolja_evaluacija:
15                  najbolja_evaluacija = vrijednost
16                  najbolje_dijete = dijete
17              alpha = max(najbolja_evaluacija, alpha)
18              if beta <= alpha:
19                  break
20
21      if bojaIgraca == CRNA:
22          najbolja_evaluacija = float('inf')
23          for dijete in pozicija.djeca:
24              vrijednost, _ = alfa_beta_podrezivanje(dijete, dubina-1, BIJELA, alpha, beta)
25              if vrijednost < najbolja_evaluacija:
26                  najbolja_evaluacija = vrijednost
27                  najbolje_dijete = dijete
28              beta = min(najbolja_evaluacija, beta)
29
30              if beta <= alpha:
31                  break
32      pozicija.evaluacija = najbolja_evaluacija
33      return najbolja_evaluacija, najbolje_dijete
```

Korektnost α-β podrezivanja

Neka je $M(v) \in \mathbb{R}$ stvarna vrijednost Minimax stabla u čvoru v. Funkcija AlphaBeta$(v, d, \alpha, \beta) \in \mathbb{R}$ računa vrijednost odsječenu na interval $[\alpha, \beta]$:

Teorem: Knuth & Moore, 1975

Za svaki čvor v i parametre pretrage $\alpha < \beta$ vrijedi identitet:

$$\text{AlfaBeta}(v, d, \alpha, \beta) = \min\left(\beta, \max(\alpha, M(v))\right)$$

Korektnost α-β podrezivanja

Neka je $M(v) \in \mathbb{R}$ stvarna vrijednost Minimax stabla u čvoru v. Funkcija AlphaBeta$(v, d, \alpha, \beta) \in \mathbb{R}$ računa vrijednost odsječenu na interval $[\alpha, \beta]$:

Teorem: Knuth & Moore, 1975

Za svaki čvor v i parametre pretrage $\alpha < \beta$ vrijedi identitet:

$$\text{AlfaBeta}(v, d, \alpha, \beta) = \min\left(\beta, \max(\alpha, M(v))\right)$$

Pokretanjem algoritma iz korijena v_0 s vrijednostima $\alpha = -\infty, \beta = +\infty$ Dobivamo:

$$\text{AlfaBeta}(v_0, d, -\infty, \infty) = \min\left(+\infty, \max(-\infty, M(v_0))\right) = M(v_0)$$

Korektnost α-β podrezivanja

Neka je $M(v) \in \mathbb{R}$ stvarna vrijednost Minimax stabla u čvoru v. Funkcija AlphaBeta$(v, d, \alpha, \beta) \in \mathbb{R}$ računa vrijednost odsječenu na interval $[\alpha, \beta]$:

Teorem: Knuth & Moore, 1975

Za svaki čvor v i parametre pretrage $\alpha < \beta$ vrijedi identitet:

$$\text{AlfaBeta}(v, d, \alpha, \beta) = \min\left(\beta, \max(\alpha, M(v))\right)$$

Pokretanjem algoritma iz korijena v_0 s vrijednostima $\alpha = -\infty, \beta = +\infty$ Dobivamo:

$$\text{AlfaBeta}(v_0, d, -\infty, \infty) = \min\left(+\infty, \max(-\infty, M(v_0))\right) = M(v_0)$$

Skica dokaza korektnosti (1/2)

Matematička indukcija po dubini stabla d:

- **Baza indukcije ($d = 0$):** Za listove oba algoritma vraćaju statičku evaluaciju $\mathit{Eval}(v) = M(v)$ u rasponu $[\alpha, \beta]$.

Skica dokaza korektnosti (1/2)

Matematička indukcija po dubini stabla d:

- **Baza indukcije ($d = 0$):** Za listove oba algoritma vraćaju statičku evaluaciju $\mathit{Eval}(v) = M(v)$ u rasponu $[\alpha, \beta]$.

- **Korak indukcije:** Pretpostavimo da tvrdnja vrijedi za sva stabla dubine $d = k$. To znači da za svako dijete w na dubini k algoritam točno računa:

$$\text{AlfaBeta}(w, k, \alpha, \beta) = \min(\beta, \max(\alpha, M(w)))$$

Skica dokaza korektnosti (2/2)

Promatramo BSO MAX čvor v čija su djeca $w_1, \ldots, w_m$ podstabla dubine k. Prava Minimax vrijednost je $M(v) = \max_i M(w_i)$.

Skica dokaza korektnosti (2/2)

Promatramo BSO MAX čvor v čija su djeca $w_1, \ldots, w_m$ podstabla dubine k. Prava Minimax vrijednost je $M(v) = \max_i M(w_i)$.

1. Algoritam redom ispituje djecu i ažurira donju među:

$$\alpha_{\text{novo}} = \max(\alpha_{\text{staro}},\ M(w_i))$$

Skica dokaza korektnosti (2/2)

Promatramo BSO MAX čvor v čija su djeca $w_1, \ldots, w_m$ podstabla dubine k. Prava Minimax vrijednost je $M(v) = \max_i M(w_i)$.

1. Algoritam redom ispituje djecu i ažurira donju među:

$$\alpha_{\text{novo}} = \max(\alpha_{\text{staro}},\ M(w_i))$$

2. Ako za neko dijete w_j zadovolji se uvjet reza $\alpha_{\text{novo}} \geq \beta$:

$$M(v) \geq M(w_j) \geq \beta$$

Skica dokaza korektnosti (2/2)

Promatramo BSO MAX čvor v čija su djeca $w_1, \ldots, w_m$ podstabla dubine k. Prava Minimax vrijednost je $M(v) = \max_i M(w_i)$.

1. Algoritam redom ispituje djecu i ažurira donju među:

$$\alpha_{\text{novo}} = \max(\alpha_{\text{staro}},\ M(w_i))$$

2. Ako za neko dijete w_j zadovolji se uvjet reza $\alpha_{\text{novo}} \geq \beta$:

$$M(v) \geq M(w_j) \geq \beta$$

3. Budući da za svaki $M(v) \geq \beta$ vrijedi:

$$\min(\beta,\ \max(\alpha,\ M(v))) = \beta$$

preostala djeca $w_{j+1}, \ldots, w_m$ mogu samo dodatno povećati vrijednost, a ne je smanjiti.

Skica dokaza korektnosti (2/2)

Promatramo BSO MAX čvor v čija su djeca $w_1, \ldots, w_m$ podstabla dubine k. Prava Minimax vrijednost je $M(v) = \max_i M(w_i)$.

1. Algoritam redom ispituje djecu i ažurira donju među:

$$\alpha_{\text{novo}} = \max(\alpha_{\text{staro}},\ M(w_i))$$

2. Ako za neko dijete w_j zadovolji se uvjet reza $\alpha_{\text{novo}} \geq \beta$:

$$M(v) \geq M(w_j) \geq \beta$$

3. Budući da za svaki $M(v) \geq \beta$ vrijedi:

$$\min(\beta,\ \max(\alpha,\ M(v))) = \beta$$

   preostala djeca $w_{j+1}, \ldots, w_m$ mogu samo dodatno povećati vrijednost, a ne je smanjiti.

Prekidom pretrage dobivena vrijednost ostaje točno β. □

Igraju li računala šah ovako i danas?

AlphaZero (Prosinac 2017) - Pregazio tadašnji Stockfish 8.
Ne koristi Minimax nego **Monte Carlova Pretraga Stabla (MCTS)**

- **Osnovna ideja:** Asimetrična pretraga vođena statistikom simulacija umjesto pretrage fiksne dubine u širinu.

Igraju li računala šah ovako i danas?

AlphaZero (Prosinac 2017) - Pregazio tadašnji Stockfish 8.
Ne koristi Minimax nego **Monte Carlova Pretraga Stabla (MCTS)**

- **Osnovna ideja:** Asimetrična pretraga vođena statistikom simulacija umjesto pretrage fiksne dubine u širinu.
- **Ključna mehanika:** *Selekcija* $\rightarrow$ *Ekspanzija* $\rightarrow$ *Evaluacija* $\rightarrow$ *Propagacija unatrag*.

Igraju li računala šah ovako i danas?

AlphaZero (Prosinac 2017) - Pregazio tadašnji Stockfish 8.
Ne koristi Minimax nego **Monte Carlova Pretraga Stabla (MCTS)**

- **Osnovna ideja:** Asimetrična pretraga vođena statistikom simulacija umjesto pretrage fiksne dubine u širinu.
- **Ključna mehanika:** *Selekcija* $\rightarrow$ *Ekspanzija* $\rightarrow$ *Evaluacija* $\rightarrow$ *Propagacija unatrag*.
- Kombinacijom s dubokim neuronskim mrežama (AlphaZero) rješava problem prevelikog faktora grananja.

Igraju li računala šah ovako i danas?

AlphaZero (Prosinac 2017) - Pregazio tadašnji Stockfish 8.
Ne koristi Minimax nego **Monte Carlova Pretraga Stabla (MCTS)**

- **Osnovna ideja:** Asimetrična pretraga vođena statistikom simulacija umjesto pretrage fiksne dubine u širinu.
- **Ključna mehanika:** *Selekcija* $\rightarrow$ *Ekspanzija* $\rightarrow$ *Evaluacija* $\rightarrow$ *Propagacija unatrag*.
- Kombinacijom s dubokim neuronskim mrežama (AlphaZero) rješava problem prevelikog faktora grananja.

Za one koji žele znati više

- C. Browne et al., *A Survey of Monte Carlo Tree Search Methods*, IEEE TCIAIG, 2012. (pregled algoritma i UCT/PUCT selekcije)
- D. Silver et al., *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*, Science, 2018. (arhitektura AlphaZero sistema)

Stockfish uzvrača udarac

Stockfish zadržava α-β Minimax pretragu po stablima. Više ne koristi tvrdo-kodirane funkcije evaluacije. Prelazi na **Učinkovito ažurabilnu neuronsku mrežu (NNUE):**[3]
Koristi malu plitku neuronsku mrežu unutar α-β pretrage.
Ključne inženjerske prednosti:

- **Inkrementalno (diferencijalno) ažuriranje:** Pri potezu se ne evaluira cijela mreža ispočetka, već samo promjene nastale pomakom jedne figure.

[3] Yu Nasu, 2018. (izvorno za Shogi, integrirano u Stockfish 12 u kolovozu 2020.)

Stockfish uzvrača udarac

Stockfish zadržava α-β Minimax pretragu po stablima. Više ne koristi tvrdo-kodirane funkcije evaluacije. Prelazi na **Učinkovito ažurabilnu neuronsku mrežu (NNUE):**[3]
Koristi malu plitku neuronsku mrežu unutar α-β pretrage.
Ključne inženjerske prednosti:

- **Inkrementalno (diferencijalno) ažuriranje:** Pri potezu se ne evaluira cijela mreža ispočetka, već samo promjene nastale pomakom jedne figure.
- **Izvođenje na CPU-u:** Koristi SIMD/AVX vektorske instrukcije i cjelobrojnu (int8/int16) aritmetiku umjesto teških GPU matrica.

[3] Yu Nasu, 2018. (izvorno za Shogi, integrirano u Stockfish 12 u kolovozu 2020.)

Stockfish uzvrača udarac

Stockfish zadržava α-β Minimax pretragu po stablima. Više ne koristi tvrdo-kodirane funkcije evaluacije. Prelazi na **Učinkovito ažurabilnu neuronsku mrežu (NNUE):**[3]
Koristi malu plitku neuronsku mrežu unutar α-β pretrage.
Ključne inženjerske prednosti:

- **Inkrementalno (diferencijalno) ažuriranje:** Pri potezu se ne evaluira cijela mreža ispočetka, već samo promjene nastale pomakom jedne figure.
- **Izvođenje na CPU-u:** Koristi SIMD/AVX vektorske instrukcije i cjelobrojnu (int8/int16) aritmetiku umjesto teških GPU matrica.
- **Brzina:** Omogućuje evaluaciju desetaka milijuna pozicija u sekundi uz pozicijsko razumijevanje na razini dubokih mreža.

[3] Yu Nasu, 2018. (izvorno za Shogi, integrirano u Stockfish 12 u kolovozu 2020.)

Hvala na pažnji!

Literatura

- C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, Ser. 7, Vol. 41, No. 314, 1950.
- D. E. Knuth, R. W. Moore, *An Analysis of Alpha-Beta Pruning*, Artificial Intelligence, 6(4): 293–326, 1975.
- J. Tromp, *Counting Chess Positions*, ICGA Journal, 44(2): 73–83, 2022.
- C. Browne et al., *A Survey of Monte Carlo Tree Search Methods*, IEEE Transactions on Computational Intelligence and AI in Games, 4(1): 1–43, 2012.
- D. Silver et al., *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*, Science, 362(6419): 1140–1144, 2018.
- Y. Nasu, *Efficiently Updatable Neural-Network-based Evaluation Functions for computer shogi*, 2018. (Stockfish integracija, 2020.)